

오늘의 작업기록 — 2026년 7월 14일

예외 → HTTP 번역 완결 — 핸들러 3개 체제

(코드는 10분짜리 복제였고, 본체는 판단 2개였던 날)

한눈에

항목	내용
목표	조각 1: 핸들러 2개 추가(없는 계좌·amount≤0) + 4연타 검증 · 조각 2: 에러 JSON 구조화
결과	조각 1 완주 ✓ (핸들러 3개 체제, 4연타 400/400/400/200) · 조각 2는 40분 규칙 발동으로 이월
커밋	e567011 "map AccountNotFoundException and invalid amount to 4xx (verified: 4xx/400/400/200)" — 1 file, +9줄
★ 판단 1	없는 계좌 = 400 (404 반론 기각 — zzz는 URL이 아니라 body 필드, 고칠 사람은 클라이언트)
★ 판단 2	amount≤0 = IllegalArgumentException (6/25 결정 소환 — 단, JDK 공용이라는 트레이드오프 인지한 채)
★ 새 개념	cd A && cmd & 의 cd는 그 줄(작업) 안에서만 유효 — 내 셸은 제자리 · grep 판정 패턴의 정밀도("@" 한 글자가 조준)
트러블 2	성급한 curl(7/12 실수 재발) · server.log 부재 소동(→ cd 스코프)
부수 확인	④를 같은 key로 두 번 → 응답 동일 = 역등성 재확인 · 세 실패가 전부 400 → 구조화(조각 2)의 필요가 눈으로 보임
다음	조각 2 (다음 세션 첫 15분): ErrorResponse record + 핸들러 3개 수정 + Content-Type 변화 관찰

어제(7/13) 지반을 파고, 오늘 그 위에 앉았다. 핸들러 코드 자체는 어제 9줄 패턴에서 예외 타입만 바꾼 복제라 10분거리였다 — 오늘 시간을 쓴 곳은 코드가 아니라 그 앞의 판단 2개였고, 정확히 그게 어제 "다음 세션 준비"에 세워둔 설계였다. 예외 번역 장(7/12~7/14)이 이걸로 닫혔다.

0. 오늘의 위치 — 3일에 걸친 한 장(章)의 완결

날	한 일	성격
7/12	잔액 부족 → 400 (핸들러 1개)	패턴 습득 — @ExceptionHandler, ResponseEntity
7/13	심화 6문답 (새 코드 0줄)	지반 파기 — 4xx/5xx 경계선, 판정 도구
7/14 (오늘)	핸들러 2개 추가 + 4연타	확장·완결 — 판정 도구의 첫 실전

배우고 → 파고 → 확장한다. 이 사이클이 한 바퀴 돈 첫 사례다.

1. 오늘의 본체 — 판단 2개

판단 1 — 없는 계좌: 404인가 400인가 → 400

어제 판정 도구 ①("이게 터졌을 때 고쳐야 할 사람은 누구인가")로 4xx까지는 즉시 확정된다 — 클라이언트가 id를 고치면 되니까. 문제는 그 안에서 어느 4xx냐였다.

후보	논리	오늘의 평가
404 NOT_FOUND	"그 자원(계좌)이 없다" — REST 관점에서 자원 부재	기각. 404의 고전적 의미는 "그 URL의 자원이 없다"인데, zzz는 URL이 아니라 body 필드로 왔다. URL(/transfer) 자체는 멀쩡히 존재하고 정상 작동했다
400 BAD_REQUEST ✓	"요청 내용이 잘못됐다" — 잘못된 id를 넣은 것은 요청의 결함	채택. 근거: "이체를 시도한 사람의 잘못이다." 서버도, URL도 정상이었고 요청에 담긴 내용이 틀렸다

이 문제는 실무에서도 갈린다(404파, 400파, 422파가 다 있다). 그래서 완료 조건이 "정답 맞히기"가 아니라 "선택 + 근거 한 줄"이었고, 그걸 충족했다. 남는 것은 답이 아니라 — 어제 만든 판정 도구를 오늘 처음 실전에 돌렸다는 사실이다.

판단 2 — amount ≤ 0: 어떤 예외인가 → IllegalArgumentException

6/25에 내린 결정의 소환이었다. 그날 "금액이 0 이하"라는 실패에 대해 도메인 예외를 새로 만들지 않고 JDK에 이미 있는 정확한 이름을 쓴다고 정했다 — 그게 IllegalArgumentException ("인자가 잘못됐다")이다.

정직 기록: 이름 소환에 실패했다. "모르겠다"가 첫 답이었고, 힌트("인자가 잘못됐다는 뜻의 JDK 예외") 한 번에 나왔다. 저장은 돼 있는데 인출 경로가 약한 상태 — 복기 훈련이 정확히 이걸 위해 있다.

★ 트레이드오프를 인지한 채 쓴다 (어제 세워둔 요구):

	도메인 예외 (InsufficientBalance 등)	JDK 공용 예외 (IllegalArgumentException)
핸들러가 잡는 범위	내가 만든 그 예외뿐 — 좁고 명확	이 컨트롤러 실행 경로에서 터지는 그 타입 전부 + 하위 타입까지
위험	없음	의도치 않은 다른 IAE까지 "이체 실패: ..."로 포장될 수 있음

지금 안전한 이유: 이 컨트롤러의 실행 경로(transfer → Bank)에서 IllegalArgumentException이 나오는 자리는 **금액 검증**뿐이다. 다른 IAE를 던지는 코드가 없다. **재검토 시점:** 이 컨트롤러에 로직·경로가 늘어날 때 — 모르고 쓰는 것과 알고 쓰는 것의 차이를 오늘 만들었다.

2. Before 증거 — 그리고 트러블 2건

부팅과 성공한 curl (실수 재발)

```
cd ~/ledger-api && ./gradlew bootRun > server.log 2>&1 &
curl -i -X POST localhost:8080/transfer ... # ← grep 판정 없이 바로 침
# curl: (7) Failed to connect to localhost port 8080 after 3 ms
```

7/12와 똑같은 실수의 재발이다. 서버가 뜨기 전(빌드 10~20초)에 curl을 쳐서 연결 거부. 같은 실수 두 번은 우연이 아니라 습관의 부재다 — "**grep "Started" 판정 → 그 다음 curl**"을 부팅 후 고정 절차로 몸에 박는다.

server.log 부재 소동 → ★ 오늘의 새 개념: cd와 &의 스코프

```
sleep 15 && grep "Started LedgerApi" server.log
# grep: server.log: No such file or directory ← ?? 근데 curl은 500을 받았다 = 서버는 살아있다
```

파일이 없는 게 아니라 **내가 다른 곳에 있었다.** 추적:

```
pwd          # → /root          ← 셸은 홈에 있다!
ls ~/ledger-api/server.log # → 존재함 ← 파일은 프로젝트 폴더에 있다
```

원인 — **cd A && cmd & 에서 &** 는 그 줄 전체를 하나의 백그라운드 작업으로 보낸다. 그 안의 **cd** 는 그 작업(서브셸) 안에서만 유효하고, 내가 앉아 있는 대화형 셸의 위치는 **바뀌지 않는다.** 그래서:

- 백그라운드 작업은 ~/ledger-api로 이동해 거기에 server.log를 만들었고
- 내 셸은 /root에 그대로 있어서 grep이 파일을 못 찾았다

터미널이 증거를 직접 보여줬다:

```
[1] Terminated cd ~/ledger-api && ./gradlew bootRun > server.log 2>&1 (wd: ~)
(wd now: ~/ledger-api)
```

(wd: ~) = 그 작업이 출발한 위치, (wd now: ...) = 지금 내 셸의 위치 — 둘이 달랐다는 기록이다. (7/12엔 이 문제가 안 보였던 이유: 그날은 셸이 **이미** ~/ledger-api에 있었다.)

교훈: 백그라운드로 띄울 땐 ① cd를 먼저 **별도 줄**로 치고 나서 &를 쓰거나 ② 판정 명령에 **절대경로**(**grep ... ~/ledger-api/server.log**)를 쓴다.

Before 확보

cd ~/ledger-api 로 제 위치에 들어간 뒤, zzz 이체:

```
HTTP/1.1 500
{"timestamp": "...", "status": 500, "error": "Internal Server Error", "path": "/transfer"}
```

없는 계좌도 어제의 잔액 부족과 똑같이 **500으로 새고 있었다** — 오늘 고칠 병증의 증거 확보.

3. 코드 — 핸들러 2개 추가 (오늘의 +9줄)

기존 핸들러(InsufficientBalance) 아래, 클래스 닫는 } 앞에:

```
@ExceptionHandler(AccountNotFoundException.class)
public ResponseEntity<String> handleAccountNotFound(AccountNotFoundException e) {
    return ResponseEntity.status(HttpStatus.BAD_REQUEST).body("이체 실패: " + e.getMessage());
}

@ExceptionHandler(IllegalArgumentException.class)
public ResponseEntity<String> handleIllegalArgument(IllegalArgumentException e) {
    return ResponseEntity.status(HttpStatus.BAD_REQUEST).body("이체 실패: " + e.getMessage());
}
```

어제 9줄 패턴에서 바뀐 것은 괄호 안의 예외 타입과 메서드 이름뿐이다. 이게 "패턴을 익혔다"의 실체다 — 두 번째부터는 복제가 된다.

완성된 TransferController 전문 (핸들러 3개 체제, 커밋 e567011)

```
package com.ledger.api;

import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;

@RestController
public class TransferController {

    private final Bank bank = new Bank();

    public TransferController() {
        bank.createAccount("a", "이름", 2100);
        bank.createAccount("b", "이름", 3500);
        bank.createAccount("platform", "플랫폼", 0);
    }

    @PostMapping("/transfer")
    public String transfer(@RequestBody TransferRequest request) {
        bank.transfer(request.fromId(), request.toId(), request.amount(), request.idempotencyKey());
        return "a: " + bank.getAccount("a").getBalance()
            + ", b: " + bank.getAccount("b").getBalance()
            + ", platform: " + bank.getAccount("platform").getBalance();
    }

    @ExceptionHandler(InsufficientBalanceException.class)
    public ResponseEntity<String> handleInsufficientBalance(InsufficientBalanceException e) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body("이체 실패: " + e.getMessage());
    }

    @ExceptionHandler(AccountNotFoundException.class)
    public ResponseEntity<String> handleAccountNotFound(AccountNotFoundException e) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body("이체 실패: " + e.getMessage());
    }

    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<String> handleIllegalArgument(IllegalArgumentException e) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body("이체 실패: " + e.getMessage());
    }
}
```

새 임포트가 0개인 이유 (셋 다 이유가 다르다)

이름	임포트 불필요한 이유
HttpStatus · ResponseEntity · ExceptionHandler	어제(7/12) 이미 임포트됨 — 오늘은 재사용
AccountNotFoundException	같은 패키지(com.ledger.api) — 같은 동네는 주소 생략
IllegalArgumentException	java.lang 소속 — String처럼 자바가 자동으로 보이게 해주는 기본 꾸러미

메시지의 출처 — 3주 전의 문장들이 HTTP로 나온다

오늘 응답에 실린 세 메시지는 전부 **mini-ledger 시절 도메인에 내가 직접 심은 문장**이다: "계좌를 찾을 수 없습니다"(6/22, null 검사를 예외로 바꿀 때), "출금액은 0보다 커야 합니다"(6/25, 금액 검증), "잔액이 충분하지 않습니다"(6/25). 도메인 수정 0줄 원칙은 오늘도 유지 — 컨트롤러는 그 문장들을 **번역만** 했다. 그리고 이것이 어제 Q5 원칙("4xx의 메시지는 친절하게 — 내가 쓴 도메인 메시지라 노출 OK")의 실전이다.

4. 판정 — grep 패턴의 정밀도 (★작은 새 교훈)

저장 후 판정에서 예상과 다른 숫자가 나왔다:

```
grep -c "ExceptionHandler" TransferController.java    # → 4   ← 3이어야 하는데?
grep -c "@ExceptionHandler" TransferController.java    # → 3   ← 정답
```

원인: 패턴 `"ExceptionHandler"` 는 핸들러 3개뿐 아니라 **import 줄**(`import ...ExceptionHandler;`)까지 센다. `@` 한 글자를 붙이는 순간 "애노테이션으로 쓰인 자리"만 조준된다.

교훈: **판정 도구 자체가 정밀하지 않으면 판정이 오염된다.** 숫자가 예상과 다를 때 "코드가 틀렸나"보다 먼저 "**패턴이 뭘 세고 있나**"를 의심할 것. (grep는 6/22부터 쓴 도구지만, 패턴의 오탐(false positive)을 실제로 만난 건 처음이다.)

```
./gradlew compileJava # → BUILD SUCCESSFUL in 36s
```

5. 검증 4연타 — 신규 2 + 회귀 2

재부팅(`kill %1` → `bootRun` → `Started ... in 4.368 seconds`) 후:

#	요청	응답	성격
①	fromId zzz (없는 계좌)	400 + <code>이체 실패: 계좌를 찾을 수 없습니다</code>	신규 — 오늘 핸들러 1
②	amount -100	400 + <code>이체 실패: 출금액은 0보다 커야 합니다</code>	신규 — 오늘 핸들러 2
③	amount 99999 (잔액 부족)	400 + <code>이체 실패: 잔액이 충분하지 않습니다</code>	회귀 — 어제 것 안 깨졌나
④	정상 이체 1000 (key: ok-2)	200 + <code>a: 1080, b: 4500, platform: 20</code>	회귀 — 멀쩡한 길 안 막혔나

설계 원리 — 왜 4개인가: 새 코드의 검증은 "새 기능이 되나"(①②) 절반 + "원래 되던 게 여전히 되나"(③④) 절반이다. 예외 핸들러는 잘못 걸리면 멀쩡한 요청까지 잡아버릴 수 있는 종류의 코드라, 회귀 확인이 특히 중요하다 — 7/12에 배운 원칙의 확장 적용.

부수 확인 2개: - ④를 같은 **key(ok-2)**로 두 번 쳤는데 응답이 완전히 동일(잔액 불변) — 핸들러를 엮어도 **역등성이 그대로 작동**함을 덤으로 재확인했다. - Content-Length가 48 / 52 / 50 / 30 — 메시지 길이가 그대로 반영된 값. 본문이 온전히 갔다는 부수 판정이다 (7/6에 16으로 배운 그 도구).

종료 절차: `kill %2` → `Exit 143` → `ss -tlnp | grep 8080` **빈 출력** = 완전 종료. (자식 java의 시차 정리는 이제 예상 범위 — 7/7·7/12에서 배운 그대로.)

커밋:

```
[main e567011] map AccountNotFoundException and invalid amount to 4xx (verified: 4xx/400/400/200)
1 file changed, 9 insertions(+)
e6d10b5..e567011 main -> main
```

6. 관찰 — 셋 다 400이다: 다음 작업의 동기가 눈에 보였다

오늘 검증 표를 다시 보면, 서로 다른 세 실패(없는 계좌 / 잘못된 금액 / 잔액 부족)가 **전부 400**이다. 이 API를 쓰는 앱 입장에서:

- **상태코드로는 세 경우를 구분할 수 없다** — 전부 400.
- 구분하려면 메시지 문자열을 파싱해야 하는데(`body.contains("잔액")` ...), 그게 정확히 어제 Q2에서 "끔찍하다"고 결론 낸 병이다 — 문구 하나 바꾸면 앱이 조용히 깨지고, 다국어에서 무너진다.

어제는 이걸 **논리**로 알았고, 오늘은 **눈**으로 봤다. 핸들러가 3개가 된 지금이 어제 정해둔 "구조화를 꺼낼 시점"이다.

조각 2 설계 (다음 세션 첫 15분 — 오늘은 40분 규칙으로 이월)

- `record ErrorResponse(String code, String message)` — TransferRequest처럼 record 한 줄. **code**는 기계용(INSUFFICIENT_BALANCE / ACCOUNT_NOT_FOUND / INVALID_AMOUNT — 6/27 enum의 대문자 상수 스타일), **message**는 사람용.
- 핸들러 3개의 `<String>` → `<ErrorResponse>` 교체.
- **검증 관련 포인트:** 응답 헤더의 Content-Type이 `text/plain` → `application/json` 으로 바뀌는 순간을 본다 — 잭슨이 record를 JSON으로 **직렬화**해주는 것, 즉 7/7에 본 마법 ④(JSON→record)의 **역방향**이 처음 가동되는 장면이다.

40분 규칙 발동 기록: 조각 1 커밋이 세션 ~40분 지점이었다. 조각 2는 핸들러 3개를 동시에 건드리는 작업이라 반쯤 하다 끊기면 **컴파일 깨진 채 방치**가 된다 — 최악의 꿈김. 규칙대로 시작하지 않았다. 완결 하나(조각 1)가 반쪽 둘보다 낫다.

7. 로드맵 위치

- 예외 → HTTP 번역 장 완결 (7/12~7/14, 핸들러 3개 + 검증 7회 누적).
- **ledger-api 현재:** `GET /health` · `POST /transfer` — 역등성 + 복식부기 + **세 가지 실패 전부 정확한 4xx로 응답**. 도메인 수정 여전히 0줄.
- **7월 잔여(17일):** 조각 2(15분거리) → m106 CodeCrafters Redis(온전한 세션 필요 — 새 환경 세팅 포함) → m107 월말 점검.
- **8월 WAL 이유 3개 유지:** 원자성 한계(7/5) · 영속성 부재(7/12) · 롤백 불가(7/13).

8. 검증 질문 (다음 복기용 — 답 적지 말 것)

1. 없는 계좌를 404가 아니라 400으로 고른 근거는? 404 쪽 반론과 그 기각 논리는?
2. `IllegalArgumentException` 핸들러의 트레이드오프는 무엇이고, 지금 안전한 이유와 재검토 시점은?
3. 오늘 새 임포트가 0개였다 — 이름 셋(`HttpStatus`류 / `AccountNotFoundException` / `IllegalArgumentException`)별로 이유를 대라.
4. `cd A && cmd &` 에서 `cd`가 내 셸의 위치를 못 바꾸는 이유는? 그날 터미널이 보여준 증거 두 가지는?
5. 백그라운드 기동 시 이 함정을 피하는 방법 둘은?

- 6. `grep -c "ExceptionHandler"`가 4를 반환한 이유는? 여기서 얻은 "판정 도구" 교훈을 한 문장으로.
- 7. 4연타의 설계 원리(신규/회귀 배분)는? 예외 핸들러에서 회귀 확인이 특히 중요한 이유는?
- 8. ④를 같은 key로 두 번 친 것이 부수로 재확인해준 것은?
- 9. 세 실패가 전부 400인 지금, 클라이언트가 구분할 유일한 방법과 그 방법의 병은? 해법(다음 세션)의 구성 요소는?
- 0. 다음 세션 검증에서 Content-Type의 어떤 변화를 관찰해야 하고, 그것은 무엇의 증거인가?

9. 회고

코드보다 판단이 큰 날이었다. 핸들러 두 개는 어제 패턴의 복제라 손이 기억했고, 실제로 막힌 곳도 없었다. 시간을 쓴 곳은 그 앞 — 404냐 400이냐, 그리고 어떤 예외 타입이냐. 어제 만든 판정 도구("고쳐야 할 사람이 누구인가")를 오늘 처음 실전에 돌렸고, 도구가 돌아간다는 걸 확인했다. 400이라는 답 자체보다 그게 남는다.

정직하게 남길 것 둘. 하나 — `IllegalArgumentException`의 이름이 바로 안 나왔다. 6/25에 내가 직접 내린 결정인데도. 힌트 한 번에 나온 걸 보면 지식이 없는 게 아니라 인출 경로가 약한 거고, 그게 정확히 복기 훈련이 있는 이유다. 둘 — 성급한 `curl`을 또 쳤다. 7/12와 똑같은 실수. 같은 실수 두 번은 우연이 아니라 절차의 부재라서, 오늘부로 "grep Started 판정 → `curl`"을 부팅 후 고정 절차로 못박는다.

수확도 예상 밖 자리에서 나왔다. `server.log`가 없다는 소동 — 알고 보니 파일이 아니라 내가 엉뚱한 곳에 있었고, 그 원인이 `&`의 스코프였다. `cd`가 백그라운드 작업 안에서만 유효하고 내 셸은 제자리라는 것. 터미널의 `(wd: ~)` 표시가 그 증거였다는 것까지. 사소한 소동 하나가 `bash`의 동작 원리 하나를 줬다. `grep`의 4 대 3도 같은 종류다 — 판정 도구 자체의 정밀도를 의심해본 첫 경험. 숫자가 이상하면 코드보다 패턴을 먼저 보라.

그리고 오늘 표가 스스로 다음 일을 가리켰다. 세 실패가 전부 400 — 어제는 "문자열 분기는 끔찍하다"를 논리로 알았는데, 오늘은 내 API가 정확히 그 상태라는 걸 눈으로 봤다. 다음 세션의 동기가 이미 서 있다는 뜻이다. 40분 규칙으로 조각 2를 넘긴 것도 아쉽지 않다 — 반쪽 둘보다 완결 하나. 예외 번역 장은 이걸로 닫혔고, 3일짜리 사이클(배우고, 파고, 확장한다)이 한 바퀴 완주됐다.

한 줄: 판정 도구를 처음 실전에 돌려 핸들러 3개 체제 완성(4연타 통과, e567011) — 예외 번역 장 완결. 덤: `cd`와 `&`의 스코프, `grep` 정밀도, 그리고 "셋 다 400"이 가리키는 다음 일.